# Lab 16: Two attempts, one analysis: fork the session, checkpoint the files

Topic 1: Agentic Architecture & Orchestration · Task 1.7: Manage session state, resumption, and forking
Runs on Simulator 2: Code Generation with Claude Code · Run in class
Source: SkillCertPro practice exam Part 1, Q5 (unofficial third-party question)

## The question

After a long plan-mode investigation, a developer saves the session ID before making changes. They want to test a minimal patch and a larger refactor from the same analysed context, but both attempts will touch overlapping files in the same working directory. Which approach best supports independent exploration without confusing conversation history with file state?

**A.** Fork the saved session once and run both attempts in the same working directory as isolated file branches.

**B.** Resume the saved session ID with the fork option for each attempt and checkpoint files before edits.

**C.** Use --continue twice from the project directory and rely on Claude Code to select the baseline automatically.

**D.** Resume the saved session ID twice without forking and ask Claude to keep each attempt separate.

## Simulate it live

```
# In the browser: open Simulator 2, then Lab 16
cd labs
uv run serve.py

# In Claude Code, from the labs folder
/lab 16

# In a terminal, from simulators/simulator-2-code-generation/
uv run casekit.py --case UC-5
```

## What you should see

Three session ids. Attempt B starts from restored files, so attempt A never leaks in.

Three session ids: the baseline and two forks that differ from it and from each other. Attempt B starts from restored files, so attempt A's edits never leak into it.

Open `simulators/simulator-2-code-generation/evidence/` and find it yourself.

## The answer

**B.** Resume with the fork option for each attempt, and checkpoint files first.

Forking fits when the valuable asset is the prior conversation: prompts, tool calls, results and Claude's analysis. A fork starts a new session (with its own ID) from a copy of the existing history, and the original stays unchanged. The boundary to remember is that a session is not a file-system snapshot . Forks editing the same working directory change the real files, which every session using that directory then sees. To revert, use file checkpointing, which tracks the file-editing tools (Write, Edit, NotebookEdit) but not arbitrary changes made through shell commands. Use resume for a specific earlier session ID, and fork when you need an independent branch. (For reference: claude --resume <id> --fork-session in the CLI, fork_session in the exam guide.)

## Why the others fail

**A.** One fork shared by two attempts is not two branches, and a fork never isolates files.

**C.** --continue resumes the most recent conversation in the directory, not necessarily the saved baseline. Running it twice does not create independent branches.

**D.** Without forking, both attempts append to the same history. Asking in plain language to “keep them separate” does not create isolation.

## Check yourself

- both attempts resume the baseline with --fork-session and get new session ids
- attempt A changed only formatDate.js
- files were checkpointed (restored) before attempt B, so a fork is not a filesystem snapshot
- attempt B touched callers as well

## Now change one thing

Set LAB_MODEL=claude-sonnet-5-5, rerun, and compare the result and the cost line in evidence/.

Third-party practice material from SkillCertPro, reconstructed from a video transcript in the Tertiary Markdown Vault. Not a real exam item. Wording may differ from the original.